KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 01

Importance of Labels & Running Pods

Topic: Pods & Labels | CKA Weekend Course | 2026-09-10

What a Pod Actually Is

- The business application always runs inside a **container** — the container itself sits inside a **Pod**, Kubernetes' atomic deployable unit.
- **IP address** and **Volume** (if any) are given to the **Pod**.
- **CPU and memory** are allocated at the **container** level, inside that pod.

Pod YAML Basics

- `kind: Pod`, and under `metadata` you give the pod a name (e.g. `webserver`).
- Inside `spec.containers`, you name the container (e.g. `nginx`) and give it an image.
- YAML is **indentation-sensitive** — tabs are NOT valid, 2 spaces = one indent level.
- Command to create from a file: `kubectl create -f mypod.yaml`.
- This file-based creation is **declarative** — it's Infrastructure as Code (IaC). Keep these YAML files in your VCS (Git) so they're reusable and reviewable later — unlike running a raw `docker run` command on the shell, which leaves nothing behind to refer to.
- `kubectl get pods -o wide` shows which node the pod landed on. Which node it lands on is decided by the **scheduler** — covered in more depth later.

> ⚠️ **Big Gotcha: kubectl delete -f <file>.yaml**
>
> If a YAML file defines multiple pods (e.g. both `webserver` and `apache`), running `kubectl delete -f mypod.yaml` deletes **all of them** — not just one. Always know what's inside a YAML file before running delete against it; this is a genuinely dangerous command on a shared manifest.

Multi-Container Pods

- One Pod can run more than one container — e.g. `nginx` + `redis` together.
- `kubectl get pods` shows `READY 2/2` when both containers in the pod are up — Kubernetes' atomic unit is the Pod, so there's no `kubectl get containers`, only pods.
- `kubectl describe pod` shows scheduling/start **events** — it does NOT tell you if the app inside is actually working. For that you need logs.

- `kubectl logs webserver` on a 2-container pod defaults to the **first container in the YAML** and prints `Defaulted container "nginx" out of: nginx, redis`.
- To see the other container's logs: `kubectl logs webserver -c redis`.

Labels — Why They Matter

- Labels are key/value pairs you deliberately add under `metadata.labels` — nothing is labeled by default.
- They identify *what* something is: which app, which client/customer, which environment.
- The real value of labels is **filtering, sorting, and bulk actions** across many objects — not just documentation.

Kubernetes' 6 recommended labels (a prefix like `abc.com/` or `networknuts.net/` is optional but good practice so your labels don't collide with anyone else's):

Label	Meaning
`.../name`	App name
`.../managed-by`	What manages it (helm, manual, etc.)
`.../version`	Version
`.../component`	Role in the architecture (frontend/backend)
`.../part-of`	The bigger logical app this belongs to
`.../instance`	Unique instance id

Multi-tier example: an app `webserver` has a `frontend` (Apache) and `backend` (MySQL/Redis). Both get `part-of: webserver` (ties them together as one app), but `component: frontend` vs `component: backend` (tells them apart).

Label Commands Used

```bash
# labels as columns
kubectl get pods -L app -L client -L env

# show all labels
kubectl get pods --show-labels

# filter: equals
kubectl get pods -l "client=xyz"

# filter: AND (comma = AND)
kubectl get pods -l "app=nginx,env=prod"

# filter: NOT equal
kubectl get pods -l "app=apache,client!=abc"

# filter: IN set (OR)
kubectl get pods -l "app in (apache,nginx)"

# bulk delete by label
kubectl delete pod -l "client=xyz"
```

Multi-Container Pod YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: webserver
spec:
  containers:
    - name: nginx
      image: nginx:1.14.2
      ports:
        - containerPort: 80
    - name: redis
      image: redis
```

✅ What I Practiced

- [ ] Created multi-container pod, checked `2/2` ready, compared `logs` vs `logs -c redis`.
- [ ] Deleted a YAML file with 2 pods defined and watched both disappear at once.
- [ ] Labeled pods for 2 different clients/environments and filtered with `-l`, `-L`, `--show-labels`.
- [ ] Applied the 6 recommended labels to a frontend + backend pod pair, filtered by `part-of`.
- [ ] Bulk-deleted all pods for one client using `kubectl delete pod -l "client=xyz"`.

Merged into the master reference notes: k8s-master-notes.md §4 (Workloads) — "Pod Internals, Multi-Container Pods & Labels" addendum.